

Diseño y elaboración de pruebas unitarias para la validación de software: MsrBack API

[Nombre del estudiante 1]

[Nombre del estudiante 2]

[Nombre del estudiante 3]

[Institución educativa]

[Programa o asignatura]

[Nombre del docente]

10 de mayo de 2026

Introducción

El aseguramiento de calidad permite identificar errores antes de que un sistema sea entregado a usuarios finales. En el ciclo de vida del software, las pruebas cumplen una función preventiva porque verifican que los componentes implementados respondan de acuerdo con los requerimientos definidos y reducen riesgos funcionales, de seguridad y operación. En esta actividad se evalúa una API desarrollada en ASP.NET Core, tomando como base el módulo de autenticación y seguridad.

El sistema seleccionado es MsrBack API, correspondiente al backend de ADSTRA TECH. La validación se concentra en pruebas unitarias aplicadas a funciones críticas relacionadas con el manejo de contraseñas y la generación de tokens JWT. Esta selección permite verificar comportamientos internos sin depender de base de datos, interfaz gráfica o servicios externos, lo cual facilita una evidencia clara, reproducible y alineada con el objetivo académico de diseñar casos de prueba documentados.

Descripción del sistema evaluado

Campo	Descripción
Nombre del sistema	ADSTRA TECH - MsrBack API
Tipo de sistema	API backend para gestión de procesos médicos, administrativos, autenticación, usuarios, citas, farmacia y catálogos.
Tecnología principal	ASP.NET Core / .NET 8 con arquitectura por capas.
Capas identificadas	MsrBack, MsrBack.Core y MsrBack.Infrastructure.
Repositorio	https://github.com/AdstraTech/Adstra_Back.git
Rama y commit revisado	Rama jorge; commit 43f10e1.
Ruta local del sistema	D:\ADSTRA\Adstra_Back\MsrBack

Campo	Descripción
Proyecto de pruebas creado	C:\Users\PC\OneDrive\Documentos\Proyecto_Pruebas_MsrBack_ADSTRA\MsrBack.Tests

Objetivo de la prueba

Validar mediante pruebas unitarias que las funciones principales del módulo de autenticación y seguridad de MsrBack API generen hashes de contraseña, verifiquen credenciales y creen tokens JWT de forma correcta ante datos de entrada válidos e inválidos.

Tipo de prueba seleccionada

Se seleccionaron pruebas unitarias porque permiten comprobar unidades específicas de código de manera aislada, rápida y repetible. Este tipo de prueba es pertinente para el módulo de autenticación porque las funciones de hash, verificación de contraseñas y generación de tokens pueden evaluarse sin ejecutar toda la API ni conectarse a la base de datos. De acuerdo con Microsoft (2025), los proyectos de prueba en .NET pueden crearse con xUnit y ejecutarse mediante el comando dotnet test, facilitando la automatización y trazabilidad del resultado.

Alcance

El alcance de la validación se limita a los componentes internos relacionados con autenticación y seguridad:

- PasswordHashHelper.HashPassword: generación de hash y salt para contraseñas.
- PasswordHashHelper.VerifyPassword: validación de contraseñas correctas e incorrectas.
- GenerateTokenHelper.GenerateTokenAsync: generación de token JWT con claims esperados.

Quedan fuera del alcance las pruebas de rendimiento, usabilidad, carga, despliegue, interfaz gráfica y acceso real a base de datos. Para estas áreas se recomiendan pruebas complementarias en fases posteriores.

Requerimientos a validar

ID	Requerimiento	Componente	Criterio de aceptación
REQ-01	La contraseña no debe almacenarse ni representarse en texto plano.	PasswordHashHelper.HashPassword	El método retorna un hash y un salt no vacíos, codificados en Base64, y el hash es diferente a la contraseña original.
REQ-02	El sistema debe reconocer una contraseña correcta.	PasswordHashHelper.VerifyPassword	El método retorna true cuando recibe la contraseña original, el hash y el salt generados.
REQ-03	El sistema debe rechazar una contraseña incorrecta.	PasswordHashHelper.VerifyPassword	El método retorna false cuando recibe una contraseña distinta a la original.
REQ-04	El sistema debe generar un token de autenticación válido.	GenerateTokenHelper.GenerateTokenAsync	El token JWT contiene claims principales como IdUser, Email, IdCompany, rol y fecha de expiración vigente.

Escenarios de prueba

Escenario	Descripción	Datos principales	Resultado esperado
ESC-01	Generación segura de credenciales	Contraseña Admin123*	Hash y salt generados correctamente.
ESC-02	Validación de contraseña correcta	Contraseña original + hash + salt	La verificación retorna true.

Escenario	Descripción	Datos principales	Resultado esperado
ESC-03	Rechazo de contraseña incorrecta	Contraseña distinta + hash + salt original	La verificación retorna false.
ESC-04	Creación de token de autenticación	Usuario QA con empresa, rol y clave secreta	Token JWT legible y con claims esperados.

Casos de prueba documentados

La actividad solicita mínimo tres casos de prueba; se documentan cuatro casos unitarios para ampliar la cobertura del módulo seleccionado.

TCU-001. Generar hash y salt de contraseña

Campo	Detalle
Requerimiento asociado	REQ-01
Objetivo	Verificar que una contraseña válida se transforme en un hash y un salt utilizables para almacenamiento seguro.
Datos de entrada	Contraseña: Admin123*.
Procedimiento	Ejecutar PasswordHashHelper.HashPassword(password). Validar que el hash y el salt no estén vacíos, que sean Base64 y que el hash sea diferente al texto plano.
Resultado esperado	Hash y salt generados correctamente; el hash no debe ser igual a la contraseña original.
Resultado obtenido	Se generó hash y salt correctamente; ambos valores se pudieron convertir desde Base64 y el hash fue diferente al texto plano.
Estado	Aprobado

TCU-002. Validar contraseña correcta

Campo	Detalle
Requerimiento asociado	REQ-02
Objetivo	Comprobar que la función de verificación acepta la contraseña original cuando se usa el hash y el salt generados.
Datos de entrada	Contraseña Admin123*, hash generado y salt generado.
Procedimiento	Convertir el salt a bytes y ejecutar PasswordHashHelper.VerifyPassword(password, hash, saltBytes).
Resultado esperado	La función retorna true.
Resultado obtenido	La función retornó true.
Estado	Aprobado

TCU-003. Rechazar contraseña incorrecta

Campo	Detalle
Requerimiento asociado	REQ-03
Objetivo	Comprobar que la función de verificación rechaza una contraseña diferente a la original.
Datos de entrada	Contraseña incorrecta Admin12345*, hash y salt generados para Admin123*.
Procedimiento	Ejecutar PasswordHashHelper.VerifyPassword(wrongPassword, hash, saltBytes).
Resultado esperado	La función retorna false.
Resultado obtenido	La función retornó false.
Estado	Aprobado

TCU-004. Generar token JWT válido

Campo	Detalle
Requerimiento asociado	REQ-04
Objetivo	Verificar que el helper de generación de token produzca un JWT con claims principales y expiración vigente.
Datos de entrada	Usuario QA con IdUser 7, Email qa@adstra.local, IdCompany 12, rol Administrador y clave secreta de pruebas.
Procedimiento	Ejecutar GenerateTokenHelper.GenerateTokenAsync, leer el token con JwtSecurityTokenHandler y validar claims principales.
Resultado esperado	El token contiene Email, IdUser, IdCompany, rol y fecha de expiración futura.
Resultado obtenido	El token se generó correctamente y los claims esperados fueron encontrados.
Estado	Aprobado

Implementación de las pruebas unitarias

Se creó un proyecto xUnit independiente llamado MsrBack.Tests dentro de la carpeta de entrega.

Este proyecto referencia los proyectos MsrBack.Core y MsrBack.Infrastructure para probar directamente los componentes existentes sin modificar el código fuente principal.

Comando de ejecución utilizado:

```
dotnet test
C:\Users\PC\OneDrive\Documentos\Proyecto_Pruebas_MsrBack_ADSTRA\MsrBack.Tests\MsrBack.Tests.cs
proj --logger "trx;LogFileName=unit-tests.trx" --results-directory
C:\Users\PC\OneDrive\Documentos\Proyecto_Pruebas_MsrBack_ADSTRA\Evidencias
```

Evidencias

La ejecución de pruebas produjo un archivo de resultados TRX y una evidencia textual. El resultado final fue: 0 pruebas fallidas, 4 pruebas aprobadas, 0 omitidas y 4 pruebas totales.

```

Evidencia de ejecucion de pruebas unitarias - MsrBack.Tests
Determinando los proyectos que se van a restaurar...
Todos los proyectos están actualizados para la restauración.
MsrBack.Core -> D:\ADSTRA\Adstra_Back\MsrBack\MsrBack.Core\bin\Debug\net8.0\MsrBack.Core.dll
MsrBack.Infrastructure -> D:\ADSTRA\Adstra_Back\MsrBack\MsrBack.Infrastructure\bin\Debug\net8.0\MsrBack.Infrastructure.dll
MsrBack.Tests -> C:\Users\PC\OneDrive\Documentos\Proyecto_Pruebas_MsrBack_ADSTRA\MsrBack.Tests\bin\Debug\net8.0\MsrBack.Tests.dll
Serie de pruebas para C:\Users\PC\OneDrive\Documentos\Proyecto_Pruebas_MsrBack_ADSTRA\MsrBack.Tests\bin\Debug\net8.0\MsrBack.Tests.dll (.NETCoreApp,Version=v8.0)
1 archivos de prueba en total coincidieron con el patrón especificado.
ADVERTENCIA: Se sobrescribirá el archivo de resultados: C:\Users\PC\OneDrive\Documentos\Proyecto_Pruebas_MsrBack_ADSTRA\Evidencias\unit-tests.trx
Archivo de resultados: C:\Users\PC\OneDrive\Documentos\Proyecto_Pruebas_MsrBack_ADSTRA\Evidencias\unit-tests.trx
Correctas! - Con error: 0, Superado: 4, Omitido: 0, Total: 4, Duración: 636 ms - MsrBack.Tests.dll (net8.0)

```

Figura 1. Evidencia de ejecución de pruebas unitarias con xUnit y dotnet test.

Análisis crítico

Posibles fallos encontrados

- El proyecto compila, pero presenta múltiples advertencias de nulabilidad y documentación XML. Estas advertencias no impiden la ejecución, aunque pueden anticipar errores por valores nulos en tiempo de ejecución si no se controlan adecuadamente.
- Las funciones de autenticación dependen de que la clave secreta JWT esté configurada correctamente. Una clave vacía o débil puede causar fallos o reducir la seguridad del sistema.
- El helper de hash acepta cualquier cadena como entrada. En una capa superior se recomienda validar que la contraseña no esté vacía y cumpla políticas mínimas de seguridad.

Riesgos del sistema

- Riesgo de seguridad: errores en verificación de contraseñas podrían permitir accesos no autorizados o bloquear usuarios válidos.

- Riesgo de configuración: un `TokenSettings:SecretToken` incorrecto afectaría el inicio de sesión y la autorización de endpoints protegidos.
- Riesgo de mantenibilidad: la ausencia de pruebas automatizadas en módulos críticos dificulta detectar regresiones durante cambios futuros.

Recomendaciones de mejora

- Integrar el proyecto de pruebas a la solución principal o al pipeline de integración continua para ejecutar dotnet test en cada cambio relevante.
- Agregar pruebas unitarias con mocks para AuthBLL, especialmente para usuario inexistente, usuario inactivo, contraseña incorrecta y restablecimiento de contraseña.
- Corregir progresivamente advertencias de nulabilidad en DTOs y servicios críticos.
- Agregar validaciones de entrada con atributos o FluentValidation para credenciales, contraseñas y datos requeridos.

Importancia del proceso de pruebas antes de la implementación final

Las pruebas previas a la implementación reducen defectos en producción y aumentan la confianza en los componentes críticos del sistema. En una API que maneja autenticación, usuarios y datos médicos, la validación temprana es especialmente importante porque los errores pueden impactar la confidencialidad de la información, la continuidad operativa y la experiencia del usuario. Las pruebas unitarias no reemplazan las pruebas funcionales o de integración, pero ofrecen una primera barrera de calidad rápida y automatizable.

Conclusiones

1. MsrBack API es un sistema adecuado para la actividad porque cuenta con una arquitectura por capas, módulos funcionales claros y lógica crítica susceptible de validación automatizada.
2. Las pruebas unitarias seleccionadas permiten validar comportamientos esenciales del módulo de autenticación sin depender de infraestructura externa.
3. Los cuatro casos ejecutados aprobaron, evidenciando que el hash de contraseña, la verificación de credenciales y la generación de token JWT funcionan según los criterios definidos.
4. Como mejora futura, se recomienda ampliar la cobertura hacia AuthBLL, controladores protegidos y pruebas de integración con base de datos de prueba.

Video pitch

El paquete de entrega incluye un guion de video pitch en la carpeta VideoPitch. El video sugerido debe durar entre dos y tres minutos y explicar el sistema seleccionado, el tipo de prueba, los casos ejecutados, los resultados obtenidos y las recomendaciones principales.

Referencias

Microsoft. (2025, 28 de octubre). Pruebas unitarias de C# en .NET mediante dotnet test y xUnit.

Microsoft Learn. <https://learn.microsoft.com/es-es/dotnet/core/testing/unit-testing-csharp-with-xunit>

Pressman, R. S. (2010). Ingeniería del software: Un enfoque práctico (7.^a ed.). McGraw-Hill.

Sommerville, I. (2005). Ingeniería del software (7.^a ed.). Pearson.